

Using the Translation App for a New SNOMED CT Language Project

A repeatable, expert-led method for creating, evaluating, and delivering a new SNOMED CT language translation.

Status draft for organisational review, 2026-08-26

Audience programme leads, terminology teams, subject-matter experts (SMEs), product owners, and technical operators

Purpose explain how the organisation can use the translation app and its reusable project template to develop a high-quality SNOMED CT translation for a new language

Contents

Executive summary

1. What the app does
2. What is needed before a project begins
 - 2.1 SNOMED CT terminology
 - 2.2 Bilingual translation pairs
 - 2.3 A subject-matter expert
 - 2.4 A base style guide
 - 2.5 Models and infrastructure
3. Creating a project from the template
4. How semantic retrieval supports translation
5. Producing the first pilot translations
6. SME review
7. Turning feedback into system improvements
8. Evolving the prompt with GEPA
9. Repeating the review cycle
10. Moving from pilots to a full deliverable

11. Making the platform available to users

- 11.1 Open-source, self-hosted distribution
- 11.2 Fully managed service with supplied inference
- 11.3 Managed service with customer-supplied models
- 11.4 Recommended product approach

12. Costs, resources, and a managed-service charging model

- 12.1 Measured inference baseline
- 12.2 Local or cloud-hosted model cost
- 12.3 GEPA prompt-optimisation cost
- 12.4 SME and project-team resource
- 12.5 Other costs a managed service must recover
- 12.6 Suggested customer charging structure

13. Roles and responsibilities

14. Durable outputs from the project

15. Operating principles

16. Project readiness checklist

17. Sources and pricing notes

Executive summary

The app supports an iterative, expert-led approach to SNOMED CT translation. It combines machine translation, semantically retrieved bilingual examples, an editorial style guide, automated quality checks, and structured SME review.

The machine produces translation drafts; the SME remains the authority for canonical terminology. Each SME review round creates durable assets: approved translations, corrected terms, editorial rulings, machine-checkable rules, and evidence about where the translation prompt still needs improvement. These assets are folded back into the system so that the next round starts from a stronger position.

The reusable `translation_process` project template installs this method into every new language project. It provides the project structure, gated plan, translation and review flows, starter rules, data-source definitions, and methodological safeguards. Teams therefore begin with a repeatable process rather than designing a translation workflow from scratch.

The same platform can be offered in two complementary ways: as an open-source, self-hosted solution that customers connect to their own models, and as a managed service in which we operate the platform and supply cloud inference. The managed option should combine a base service charge with metered usage and separately scoped optimisation or human services so that inference, operations, support, and SME costs are all recoverable.

At a high level, the cycle is:

```
graph TD; A[Create and configure the language project] --> B[Load SNOMED CT terminology and bilingual translation pairs]; B --> C[Build a semantic index and supply a base style guide]; C --> D[Translate a representative pilot subset and run automated QA]; D --> E[SME rates the drafts, supplies canonical terms, and gives rulings]; E --> F[Ingest feedback into gold data, rules, the style guide, and the pool]; F --> G[Optimise the prompt, including GEPA when its prerequisites are met]; G --> H[Translate a fresh subset and repeat the SME round]; H --> I[Promote the full deliverable after quality gates and SME sign-off];
```

Create and configure the language project

↓

Load SNOMED CT terminology and bilingual translation pairs

↓

Build a semantic index and supply a base style guide

↓

Translate a representative pilot subset and run automated QA

↓

SME rates the drafts, supplies canonical terms, and gives rulings

↓

Ingest feedback into gold data, rules, the style guide, and the pool

↓

Optimise the prompt, including GEPA when its prerequisites are met

↓

Translate a fresh subset and repeat the SME round

↓

Promote the full deliverable after quality gates and SME sign-off

The process repeats until the evidence shows that the target quality has been reached. The stopping decision is made through explicit quality gates and SME sign-off, not simply because a fixed number of rounds has been completed.

1. What the app does

The app is a translation drafting, evaluation, and quality-assurance environment. It is designed to make translation work reproducible and to focus scarce SME time where it has the greatest value.

For each project, it records:

- the SNOMED CT release and domain being translated;
- the bilingual reference material used at translation time;
- the prompt, model, style guide, and rules used for each run;
- the translations and automated quality findings;
- the sample sent to the SME and the feedback returned;
- the canonical, adjudicated translations created from SME decisions;
- the changes made between rounds and the evidence supporting them; and
- the quality gates used to approve the next stage or final delivery.

This creates a traceable chain from source terminology to the final translated release. A result should always be attributable to a tracked run with known code, data, prompt, and configuration versions.

The app does not replace a terminologist or clinical SME. Its role is to produce useful drafts, identify likely risks, organise review, and convert expert feedback into reusable translation policy.

2. What is needed before a project begins

A project should not move into translation until its gating inputs are available.

2.1 SNOMED CT terminology

The organisation must supply the licensed SNOMED CT content to be translated, normally in RF2 format. This may include the International Edition and, where relevant, a national extension.

The release date, edition, modules, language reference sets, and any relationship files used for hierarchy-aware checks must be recorded. SNOMED CT licensing and distribution requirements continue to apply to the source, intermediate artefacts, and translated outputs.

The first project scope should be a coherent clinical domain or hierarchy slice rather than the entire terminology. A bounded scope makes early quality measurable and gives the SME a manageable body of concepts from which to establish language conventions.

2.2 Bilingual translation pairs

The translation agent benefits substantially from existing pairs of source-language and target-language terms. Suitable sources may include:

- an existing SNOMED CT national extension;
- previously adjudicated organisational terminology;
- approved clinical dictionaries or mappings;
- relevant coding and billing vocabularies; and
- translations produced and approved during earlier rounds of the project.

Every source must have clear provenance and suitable reuse rights. Raw source data is preserved; cleaning and curation produce new versioned datasets rather than overwriting it.

A greenfield language may have little or no suitable bilingual material. In that case, the project needs a deliberate seed-pool activity: the SME or terminology team approves an initial set of representative translations before the first production-style run. Starting with an empty pool is a valid project state, but it is not sufficient for the retrieval-assisted cascade described below.

2.3 A subject-matter expert

An SME who is fluent in the target language and understands clinical terminology must be committed to several review rounds. The SME is responsible for deciding canonical forms, rating draft quality, resolving language-policy questions, and signing off the final result.

Continuity is valuable because editorial policy develops across rounds. The process also records supersession explicitly: a later SME ruling may replace an earlier one without erasing the history of either decision.

2.4 A base style guide

An existing organisational or national style guide can be supplied at project creation. If none exists, the project starts with a short seed guide covering universal matters such as script, spacing, punctuation, abbreviations, and output format.

The initial guide should not invent detailed language policy. Those decisions should emerge from evidence and SME rulings, then be incorporated into later guide versions.

2.5 Models and infrastructure

The operator selects the translation model or model cascade, embedding model, semantic index, and any frontier-model escalation service. Model choice and configuration are versioned as part of each run.

3. Creating a project from the template

The operator creates a language project in the app and supplies its basic configuration: language code and name, translation direction, terminology and data locations, model choice, style-guide location, exemplar-pool source, and the SNOMED CT relationship file used for structural checks.

The `translation_process` template then installs:

- a problem structure covering foundations, baseline translation and QA, SME feedback rounds, editorial rulings, prompt optimisation, and delivery;
- a dependency-ordered plan with explicit gates;
- flows for translation, QA, rule-based repair, review packaging, feedback ingestion, tier analysis, gold merging, absorption testing, and exemplar-pool curation;
- source definitions for the domain terms and SME-adjudicated data;
- starter hygiene and pool-curation rule files; and
- standing methodological conclusions, including SME locking, evaluation blinding, provenance, and versioning requirements.

The project initially shows which inputs are available and which gates remain blocked. This is intentional: the template exposes missing prerequisites rather than allowing an apparently successful run with inadequate reference data.

4. How semantic retrieval supports translation

The bilingual translation pairs are curated, deduplicated, and indexed semantically. The index stores a numeric representation, or embedding, of each source-language term alongside its approved target-language translation and provenance.

At translation time:

1. The English SNOMED CT term is used to search the semantic index.
2. The most relevant bilingual examples are retrieved.
3. Those examples are added to the translation agent's context together with the source term, style guide, applicable rules, and available SNOMED CT attributes.
4. The agent produces a target-language draft and records its translation context.

This is more useful than a simple exact dictionary lookup. Semantically similar concepts can provide relevant terminology even when the source wording is not identical.

Retrieved examples are contextual evidence, not automatic authority. The current style guide, hard rules, and adjudicated SME translations take precedence over noisy or superseded examples. Pool-curation rules remove sources that teach known-bad forms, while retaining the raw source and its provenance.

Evaluation must also prevent leakage. A concept being scored must not retrieve its own approved translation, and evaluation rows must be separated appropriately from prompt-optimisation and exemplar data.

5. Producing the first pilot translations

The first run should translate a small, representative subset of the chosen domain. The subset should cover different SNOMED CT hierarchies, term structures, levels of complexity, and expected translation difficulty; it should not consist only of easy or high-frequency concepts.

The pilot establishes a baseline for:

- draft translation quality;
- model agreement or uncertainty;
- retrieval quality and gaps in the exemplar pool;
- style-guide and glossary gaps;
- recurring terminology or word-order problems; and
- the usefulness of automated quality signals.

The platform can use a confidence-routed model cascade. Several drafts from a local model are compared: concepts with strong agreement can retain the local result, while disagreement triggers escalation to a stronger model. This provides a useful draft-confidence signal as well as controlling cost.

Before SME review, the app runs automated checks such as:

- required-script and forbidden-output rules;
- preservation of numbers, qualifiers, routes, and other modifiers;
- suspicious transliteration or untranslated text;
- hierarchy consistency; and
- collisions where different SNOMED CT concepts have received the same target-language term.

These checks prioritise review; they do not determine whether a clinical translation is canonical.

6. SME review

The app packages a structured review workbook or equivalent review artefact. A typical mandatory sample contains approximately 100–120 concepts, adjusted for the language, domain, and available SME time.

The sample should be blinded and stratified across the app's review-priority tiers. The SME does not see the risk tier while rating the translations. This allows the project to measure whether its prioritisation is genuinely finding poorer translations rather than merely appearing plausible to the project team.

For each reviewed concept, the SME should provide:

- a quality rating;
- the canonical target-language translation when the draft is not canonical;
- optional accepted synonyms or variants;
- an error category or short explanation where useful; and
- any broader editorial ruling that should apply to similar concepts.

The exact labels can be adapted to organisational policy. A practical scale distinguishes at least:

Rating	Meaning	Expected SME action
Perfect / canonical	The draft is the preferred canonical form as written.	No correction required.
Acceptable	Clinically and linguistically valid, but not necessarily the preferred canonical wording.	Record a preferred form or accepted synonym if useful.
Partial	The intended concept is recognisable, but one or more material changes are required.	Supply the corrected canonical translation and, ideally, the error type.
Wrong / unacceptable	The draft expresses the wrong concept or is unsafe or unusable.	Supply a canonical correction and enough context to diagnose the cause.

The review pack should also include a small number of adjudication questions. These are cross-cutting policy decisions, such as the preferred rendering of an anatomical structure, modifier order, abbreviation handling, or whether a particular source distinction must be explicit. One clear ruling can

improve hundreds of future translations.

7. Turning feedback into system improvements

The returned review is ingested through a tracked flow. It is not manually copied into the current deliverable. Ingestion validates the workbook, normalises ratings, retains SME notes, and emits a versioned review dataset.

The project team then converts feedback into the most durable form available:

1. **Adjudicated gold:** canonical translations and approved variants become the authoritative evaluation set. Machine output can never overwrite these rows.
2. **Hard rules:** deterministic requirements become machine-checkable validations and, where safe, minimal-substitution repairs.
3. **Style-guide changes:** terminology preferences, glossary entries, word-order patterns, and editorial guidance become prompt instructions.
4. **Pool-curation changes:** examples that teach a superseded or unsuitable form are excluded from future retrieval, while newly approved pairs can be added.
5. **Adjudication records:** open questions, rulings, implementation status, and superseded decisions remain traceable.

Where a new ruling conflicts with an older SME-approved translation, the older row is placed on a confirmation list. It is not silently changed and is not allowed to remain invisibly inconsistent with current policy.

The reviewed subset can then be re-translated as an absorption test. This asks whether the updated guide, rules, and reference material cause the system to reproduce the SME's decisions more consistently. Absorption is useful evidence that feedback has been encoded, but it is not evidence of generalisation because the same concepts informed the changes. Generalisation is measured on fresh, held-out concepts in the next round.

8. Evolving the prompt with GEPA

GEPA is the app's automated prompt-optimisation stage. It proposes and evaluates prompt changes using the growing body of SME-adjudicated examples and feedback.

GEPA is most valuable after straightforward terminology decisions have already been encoded as rules, style-guide entries, or pool changes. It should focus on residual problems that are genuinely prompt-sensitive, such as compositional phrasing or word order, rather than rediscovering a glossary through expensive model calls.

Before GEPA is used for a promotion decision, the project needs:

- enough SME-adjudicated examples to create separate optimisation and evaluation data;
- a leakage-free train, development, and held-out test arrangement;
- evaluation measures shown to align with the SME's acceptable/partial boundary;
- frozen hard rules that candidate prompts are not allowed to violate; and
- a promotion gate requiring the candidate prompt to outperform the current prompt on held-out data.

String metrics such as exact agreement and chrF can support comparison, but they do not replace SME judgement. Valid synonyms and acceptable variants may differ textually from a single canonical reference. The project should validate its automated metrics against human ratings and retain SME sign-off as the final authority.

The winning prompt is versioned and promoted only when the evidence supports it. An experiment that does not beat the existing prompt is still retained as research evidence but is not deployed.

9. Repeating the review cycle

After feedback has been absorbed and any prompt improvement has passed its gate, the app produces a new translation subset for SME review. Previously adjudicated concepts are excluded from the new blinded sample so that the next round tests generalisation on fresh material.

Each round follows the same controlled loop:

1. select and translate a representative subset;
2. run automated QA and assign review priority;
3. package a fresh blinded sample;
4. collect ratings, canonical corrections, and adjudication rulings;
5. ingest and analyse the review;
6. update gold data, rules, the style guide, the exemplar pool, and—when justified—the prompt;
7. test absorption and held-out improvement; and
8. audit the result before starting the next round.

The process continues until quality stabilises at the required level. Early rounds usually discover high-volume terminology and formatting conventions. Later rounds tend to concentrate on rarer terms, structural language problems, and inconsistent or genuinely ambiguous source concepts.

For planning purposes, a domain project commonly needs several rounds rather than one. Experience from the Korean imaging project suggests planning for approximately three to five rounds, with a standard review round requiring a few hours of SME time. Greenfield languages normally need additional seeding and may need more rounds.

10. Moving from pilots to a full deliverable

Once pilot evidence is strong enough, the current approved configuration is used for a broader or full-domain translation run. Automated checks and targeted SME sampling continue; increasing volume does not remove the need for review.

The final deliverable should not be promoted until agreed gates are satisfied. At minimum:

- all source and configuration versions are recorded;
- the exemplar pool and semantic index are versioned and reproducible;
- the prompt, style guide, and rules are frozen for the release candidate;
- the blinded-sample error rate meets the project's target;
- there are no unresolved blocker findings in unreviewed machine translations;

- adjudicated translations are identical to the current gold set;
- outstanding supersession and terminology-policy questions are resolved or explicitly accepted;
and
- the SME provides formal sign-off.

The app retains the run identifiers and content digests required to rebuild the release candidate and its review pack.

11. Making the platform available to users

There are two primary ways to make the translation platform available, plus a useful hybrid configuration. They can share the same codebase and project template; the difference is who operates the infrastructure, supplies the models, carries the inference cost, and supports the user.

Delivery model	Who operates the app?	Who supplies inference?	Best suited to
Open-source, self-hosted	The customer	The customer, using local models or its own API accounts	Technically capable organisations, strict data-residency requirements, and users wanting maximum control
Managed service	Us	Us, through cloud-hosted or managed model services	Customers wanting the quickest route to a supported service without ML infrastructure
Managed service with customer models	Us	The customer connects an approved endpoint or account	Customers that want managed operations but already have a model contract or mandated cloud platform

11.1 Open-source, self-hosted distribution

Under this model, users receive the complete application, the `translation_process` template, deployment material, documentation, and the language-plugin mechanism. They install it in their own environment and connect the models of their choice.

The model interface is intentionally configurable. A customer could use:

- an open-weight translation model running on its own GPU infrastructure;
- a model hosted in its existing cloud account;
- a commercial model API for uncertain or difficult concepts; or
- a confidence-routed combination of local and commercial models.

The customer would be responsible for:

- obtaining and managing its licensed SNOMED CT release;
- supplying bilingual translation pairs and confirming their reuse rights;
- providing the target-language SME;
- deploying the app, vector database, object storage, embedding service, and model endpoints;
- operating security, backups, monitoring, upgrades, and incident response; and
- paying its own infrastructure and model-provider costs.

This option gives customers the strongest control over data location, provider choice, and cost. It also means that deployment quality will vary. Good documentation, supported reference configurations, automated health checks, and a clear compatibility policy for models and plugins would be important parts of an open-source release.

Open source does not make SNOMED CT content or third-party bilingual data freely redistributable. Each user remains responsible for the applicable terminology and data licences. SNOMED International's current vendor guidance says that organisations developing or distributing products or services containing SNOMED CT generally need the appropriate Affiliate or sub-licensing arrangements; national-extension terms may add further requirements. Legal review is therefore required before choosing the software licence or bundling any terminology data.

Possible organisational support around the open-source edition could include:

- a public issue tracker, documentation, and community discussion;
- paid installation and onboarding;
- enterprise support and maintenance subscriptions;
- validated deployment images for common cloud platforms; and
- paid terminology-project services without supplying the inference platform.

11.2 Fully managed service with supplied inference

Under the managed-service model, we host the application and its supporting services, operate the semantic index, connect and pay for the approved inference providers, monitor runs, and provide upgrades and support. The customer supplies its licensed terminology, bilingual sources, style guide, and SME access through a controlled onboarding process.

This is the lowest-friction route for a customer. It provides a consistent configuration and lets us measure quality and usage across projects. It also creates responsibilities that do not exist in the self-hosted edition:

- we become responsible for availability, monitoring, backups, recovery, and support;
- we must meter model, storage, indexing, and platform use accurately;
- we carry model-price changes, failed-run costs, and customer credit risk;
- data-processing agreements, retention, deletion, security, and data-residency controls are required;
- model-provider terms must permit the proposed terminology processing; and
- service levels and the boundary between automated drafting and SME responsibility must be explicit.

The managed service could use public model APIs, managed cloud model platforms such as Amazon Bedrock, Google Vertex AI or Microsoft Foundry, or a model deployed into our own cloud account. A routed architecture is commercially useful: a lower-cost or open model handles the routine bulk work, while a stronger endpoint is used only where model disagreement indicates uncertainty.

Customers should not need to understand token accounting. We should meter tokens and cloud resources internally but present a comprehensible commercial unit externally—for example translated-concept runs, project rounds, or an included usage allowance with overage charges.

11.3 Managed service with customer-supplied models

A hybrid option allows a managed customer to connect an approved model endpoint in its own cloud or provide credentials for an existing account. We still operate the workflow, semantic index, review cycle, and QA system, while the customer holds the inference contract.

This can help with procurement, data residency, or a mandated cloud strategy, but it introduces a shared-support boundary. The service contract must say who diagnoses model failures, rate limits, endpoint changes, and output-quality regressions.

11.4 Recommended product approach

The strongest overall proposition is **open core with an optional managed service**:

1. publish the engine, project template, plugin interfaces, and deployment guidance so customers can self-host and connect their own models;
2. offer a supported managed service for customers that value rapid adoption and predictable operations;
3. make customer-supplied endpoints a managed-service configuration rather than a separate product; and
4. charge primarily for the maintained workflow, quality assurance, provenance, support, and expert process—not merely for reselling tokens.

This approach serves both residency-constrained organisations and customers without ML infrastructure, while keeping the project template and quality method consistent across deployments.

12. Costs, resources, and a managed-service charging model

The cost of a translation project has two very different parts:

- **machine and platform costs**, which scale with concepts, model calls, optimisation experiments, storage, and service operation; and
- **human expert costs**, which scale with review rounds, corrections, adjudication questions, and project support.

SME availability is usually the binding resource. API inference is visible and easy to meter, but it is not normally the largest component of the end-to-end project.

12.1 Measured inference baseline

The Korean imaging project provides a useful planning anchor. One full run translated 5,012 concepts and escalated 2,253 concepts—approximately 45%—to the stronger model. The escalated requests consumed 16.87 million input tokens and approximately 33,000 output tokens. The short output is inexpensive; the retrieved context and style guide make input the main cost.

The direct API formula is:

API cost =

(input tokens ÷ 1,000,000 × input price)

+ (output tokens ÷ 1,000,000 × output price)

Illustrative direct costs for the measured full pass are shown below. They use representative list-price bands visible on official provider pricing pages on 26 August 2026 and exclude negotiated discounts, taxes, cloud-platform charges, and our service margin.

Illustrative API rate per million tokens	Calculated direct cost for the measured 5,012-concept pass
\$1 input / \$6 output	approximately \$17
\$2.50 input / \$15 output	approximately \$43
\$3 input / \$15 output	approximately \$51
\$5 input / \$30 output	approximately \$85

These figures apply the full input rate to every measured token and are therefore conservative where prompt caching receives a discount. The measured run reported substantial cached input. Provider rates, model names, region premiums, context-length rules, and cache pricing change over time, so a production quotation must recalculate from current prices.

A typical project may need two or three broad translation passes: a baseline plus one or two re-translations after feedback has been absorbed. Small 100–120-row pilot or absorption runs cost only a fraction of a full pass. Deterministic QA and rule-based repair usually require no paid model call.

12.2 Local or cloud-hosted model cost

The confidence-routed cascade can run its bulk samples on an open-weight model. In the measured project, 5,012 concepts × five local samples took approximately 23 minutes on one GPU workstation. This replaces a large number of paid API calls with GPU time.

For a managed service, the equivalent cost depends on whether GPU capacity is:

- purchased on demand for each run;
- reserved for predictable usage;
- shared across multiple customer projects; or
- avoided by using a low-cost managed endpoint instead.

The commercial model should use measured GPU-hours multiplied by the current regional cloud rate, including idle time and failed or repeated jobs. A shared queued service is likely to be more economical than a permanently running GPU for each customer.

The semantic index also has a compute cost, but it is mainly an onboarding or pool-versioning cost. The measured fresh index of roughly 500,000 pairs took about ten minutes on local compute, and unchanged embeddings can be reused across later pool versions.

12.3 GEPA prompt-optimisation cost

GEPA evaluates many prompt candidates and should have its own budget and approval gate. Its cost depends on the number of candidates, development-set size, reflection model, evaluation model, and stopping rule.

As a rough planning envelope, a campaign may consume about 10–50 times the model budget of one evaluation pass. At the illustrative API rates above, an initial controlled allowance in the **hundreds to low thousands of US dollars** is reasonable, with a hard spend cap and early stopping. This is not a quotation: the operator should show the proposed candidate count, dataset size, model rates, and maximum token budget before each campaign.

GEPA is occasional optimisation work, not a cost attached to every translated concept. It should be run only when there is enough adjudicated data and simpler rule, guide, and pool changes have saturated.

12.4 SME and project-team resource

The standard review sample is approximately 100–120 concepts. Planning assumptions from the completed Korean rounds are:

Activity	Planning assumption
Rating-only review	approximately 2–4 concepts per minute
Review with written canonical corrections	approximately 1–2 concepts per minute
One standard review round	approximately 2–4 SME hours, including adjudication questions

Activity	Planning assumption
Rounds for a roughly 5,000-concept domain	approximately 3–5 rounds
Total SME effort	approximately 10–20 hours for a language with usable reference data
Greenfield allowance	plan roughly 50% additional SME effort and one or two additional rounds
Operator effort	approximately one setup week, then 1–2 operator days per round

The financial plan should apply the organisation's loaded rates:

```
human delivery cost =
  SME hours × loaded SME hourly rate
+ operator days × loaded operator day rate
+ project, governance, security, and support time
```

The customer may supply its own SME, contract one through us, or use a mixed arrangement. SME services should be shown separately because language, clinical domain, availability, and market rates vary substantially.

12.5 Other costs a managed service must recover

Inference is only one line in the managed-service cost base. Pricing also needs to cover:

- service onboarding, project configuration, and data validation;
- application compute, database, vector storage, object storage, backups, and network traffic;
- monitoring, logging, support, incident management, and service-level commitments;
- security reviews, vulnerability management, identity, tenant isolation, and audit evidence;
- software maintenance, model qualification, regression testing, and provider migrations;
- failed and repeated runs, free trials, non-payment, and model-price volatility;
- terminology licensing administration and data-governance work where applicable;
- customer success, invoicing, tax, procurement, and contract management; and
- a contingency and margin appropriate to the service obligations.

These costs should be measured during one or more service pilots before publishing a fixed public price.

12.6 Suggested customer charging structure

A clear managed-service price can combine:

1. **Onboarding fee** — project setup, data validation, source registration, style-guide preparation, and initial index build.
2. **Platform subscription** — hosted application, storage allowance, maintenance, security, support, and access to the maintained workflow and QA capability.

3. **Usage allowance and overage** — a number of translated-concept runs or project rounds, with additional use billed at a published rate. Internally this must reconcile to tokens, GPU-hours, and cloud resources.
4. **Optimisation package** — optional GEPA campaign with an agreed dataset, models, maximum spend, and promotion report.
5. **Professional services** — operator-led curation, custom integrations, additional QA, migration, training, or project management.
6. **SME services** — separately priced when we source or subcontract the reviewer.

One possible commercial formula is:

```
customer charge =
  onboarding
+ subscription for the service period
+ metered usage above the included allowance
+ optional optimisation and professional services
+ SME services, where supplied
+ applicable taxes
```

Billing purely at cost-per-token would be difficult for customers to predict and would underprice the workflow, assurance, and support. A concept- or round-based customer unit is easier to budget, while internal token and GPU metering protects the service from cost drift.

Before launch, the organisation should decide whether it is willing to become a data processor, which territories and cloud regions it will support, what availability it will promise, how model-price changes can be passed through, and whether customers retain or contribute language assets created through the service.

13. Roles and responsibilities

Role	Main responsibilities
Project or programme lead	Defines scope and quality targets, confirms licensing and governance, secures SME availability, and owns gate decisions.
Terminology SME	Rates samples, supplies canonical terms, approves synonyms, resolves editorial questions, and signs off the deliverable.
Translation operator / engineer	Provisions the project, prepares and indexes data, runs flows, packages reviews, encodes SME feedback, and maintains reproducibility.
Data or governance owner	Confirms that SNOMED CT content and bilingual sources may be used, stored, and distributed as proposed.
Product owner or service manager	Agrees how approved translations will be published, maintained, and updated for later SNOMED CT releases.

The project lead should agree decision rights at the outset. In particular, the terminology SME must be recognised as the authority for canonical target-language terminology, while technical staff remain responsible for faithfully encoding and testing those decisions.

14. Durable outputs from the project

A successful project produces more than a translated spreadsheet. Its reusable outputs are:

1. a versioned SNOMED CT translation deliverable;
2. an adjudicated gold set with canonical terms, accepted variants, and supersession history;
3. a curated, licensed bilingual exemplar pool and its reproducible semantic index;
4. a target-language style guide and glossary;
5. machine-checkable language and quality rules;
6. a versioned translation prompt and its evaluation evidence;
7. SME review packs, returned feedback, and adjudication records; and
8. a provenance trail linking each promoted result to its source data, configuration, and tracked runs.

These assets make subsequent domains and later SNOMED CT releases cheaper to translate. The style guide, exemplar pool, gold set, and rules compound across the life of the language service.

15. Operating principles

The following principles protect quality and auditability:

- **SME decisions are authoritative.** Machine-generated text never overwrites an adjudicated translation.
- **Supersession is explicit.** A newer SME ruling may replace an older one, but the change and its rationale remain visible.
- **Canonical forms come from adjudicated sources.** Frequency in a noisy bilingual pool does not establish authority.
- **Derived data is never edited in place.** Curation, repair, and gold merging create new versioned artefacts.
- **Metrics come from tracked runs.** Ad hoc calculations are not used as formal decision evidence.
- **Experiments are blinded and leakage-free.** Evaluation concepts cannot reveal their own approved translations through retrieval or optimisation data.
- **Deterministic improvements come first.** Rules, glossary changes, and pool fixes are applied before automated prompt optimisation.
- **Prompt changes require held-out evidence.** A plausible prompt is not promoted merely because it looks better.
- **SME sign-off remains the final delivery gate.** Automated metrics and detectors support, but do not replace, expert approval.

16. Project readiness checklist

Before approving the first pilot run, confirm that:

- [] the project scope and target SNOMED CT release are agreed;
- [] licensing and information-governance checks are complete;
- [] the RF2 files and required relationship data are available;
- [] the bilingual sources are approved, versioned, and traceable;
- [] a usable seed exemplar pool exists, or a seed-pool activity is planned;
- [] an SME is assigned and review time is reserved;
- [] the rating scale and definition of canonical translation are agreed;
- [] a base style guide exists, even if intentionally minimal;
- [] models, compute, and any external API use are approved;
- [] the initial domain slice and representative pilot sample are defined;
- [] evaluation leakage controls are in place; and
- [] the quality thresholds and final sign-off route are documented.

Once these items are satisfied, the project template provides the operational structure for running and repeating the translation-review-improvement cycle.

17. Sources and pricing notes

Internal process and resource figures come from tracked runs in the 5,012-concept Korean imaging project and are planning anchors, not guarantees for another language or domain.

External pricing and licensing references were checked on 26 August 2026:

- [OpenAI API pricing](#) — model-specific input, cached-input, and output token rates.
- [Anthropic model price list](#) — list prices across direct API and major cloud platforms.
- [Amazon Bedrock pricing](#) — provider- and model-dependent managed inference, including batch and on-demand options.
- [SNOMED International vendor licensing guidance](#) — Affiliate, sub-licence, national-licence, and distribution considerations.
- [SNOMED International: Get SNOMED CT](#) — current access and licensing routes for Member and non-Member territories.

Provider prices and licensing terms can change. Any customer proposal must refresh these inputs and receive commercial, legal, security, and information-governance review.